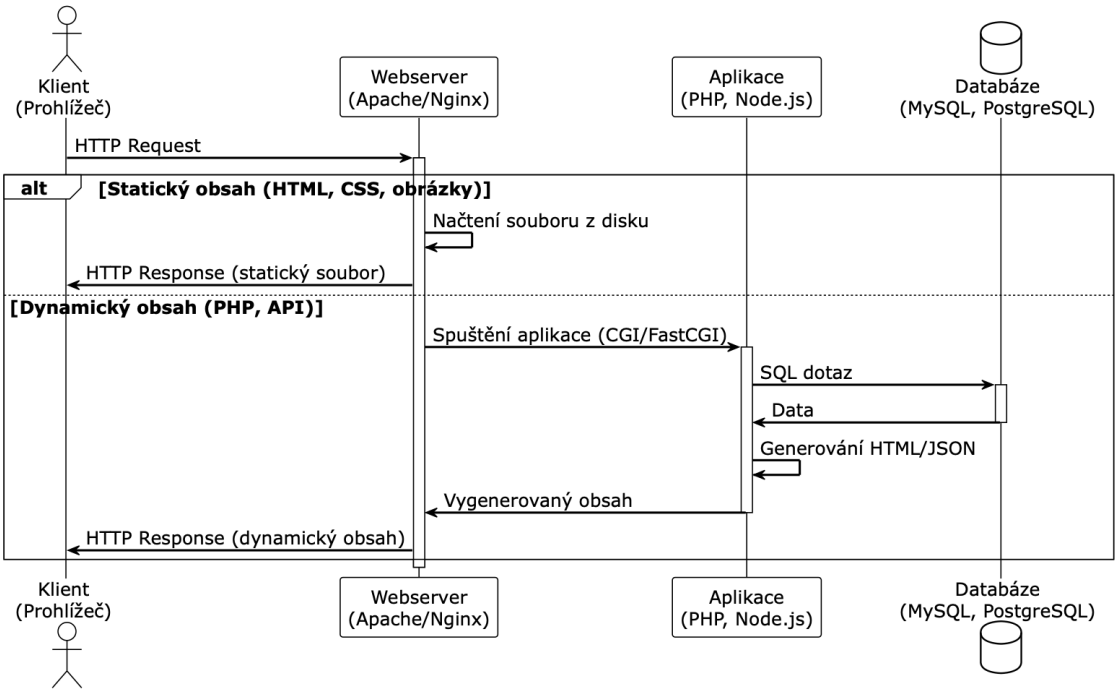
Webový server - konfigurace, cachování

Webový server

- Je software
- Naslouchá na HTTP/HTTPS portech (obvykle 80, 443)
- Přijímá požadavky od klientů (prohlížeče, API)
- Odesílá odpovědi (HTML, JSON, soubory)
- Implementuje HTTP protokol



Role webového serveru v architektuře

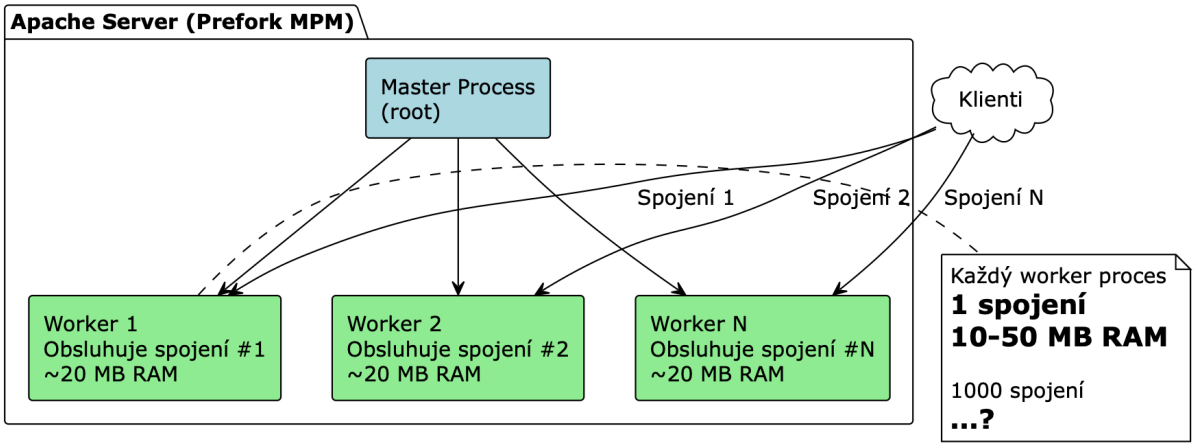
- **Tři hlavní funkce webového serveru:**

  1. **Servírování statických souborů**
     - HTML, CSS, JavaScript, obrázky, videa
     - Přímé čtení z disku a odeslání klientovi
     - Nejrychlejší operace
  2. **Spouštění dynamického obsahu**
     - PHP, Python, Node.js, Ruby přes CGI/FastCGI
     - Server spustí aplikaci, ta vygeneruje odpověď
     - Pomalejší, ale flexibilnější
  3. **Reverse proxy**
     - Přeposílání požadavků na backend servery
     - Load balancing, caching, SSL terminace
     - Moderní architektura mikroslužeb
- **Praktická ukázka -> identifikace serveru**

Testování webserveru pomocí curl

```
# Zjištění typu serveru z HTTP hlaviček
curl -I https://example.com

# Typická odpověď:
HTTP/2 200
server: nginx/1.18.0
content-type: text/html
...

# Nebo Apache:
HTTP/1.1 200 OK
Server: Apache/2.4.41 (Ubuntu)
...
```

- **Apache:**
  - Vytvořen v roce 1995
  - Nejpopulárnější webserver světa (1996-2019)
  - Open source (Apache License 2.0)
  - Cross-platform (Linux, Windows, macOS)
  - **Použití:**
    - WordPress hosting
    - Sdílený hosting
    - Legacy aplikace
  - **Apache používá Multi-Processing Modules (MPM):**
    1. **<u>Prefork MPM:</u>**
       - Jeden proces per spojení
       - Žádná vlákna -> maximální stabilita
       - Vysoká spotřeba paměti
    2. **<u>Worker MPM:</u>**
       - Kombinace procesů a vláken
       - Nižší spotřeba zapěti než Prefork
       - Vyžaduje thread-safe knihovny
    3. **<u>Event MPM:</u>**
       - Lepší pro keep-alive spojení
       - Asynchronní zpracování některých operací
       - Doporučení pro Apache 2.4+



- **Konfigurace:**
  - **Virtual host (definuje web na serveru):**

Základní konfigurace virtuálního hostu

```
<VirtualHost *:80>
    ServerName www.example.com
    ServerAlias example.com
    DocumentRoot /var/www/example

    <Directory /var/www/example>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/example-error.log
    CustomLog ${APACHE_LOG_DIR}/example-access.log combined
</VirtualHost>
```

  - **mod_rewrite (pro přepisování URL):**

URL přepisování pomocí mod_rewrite

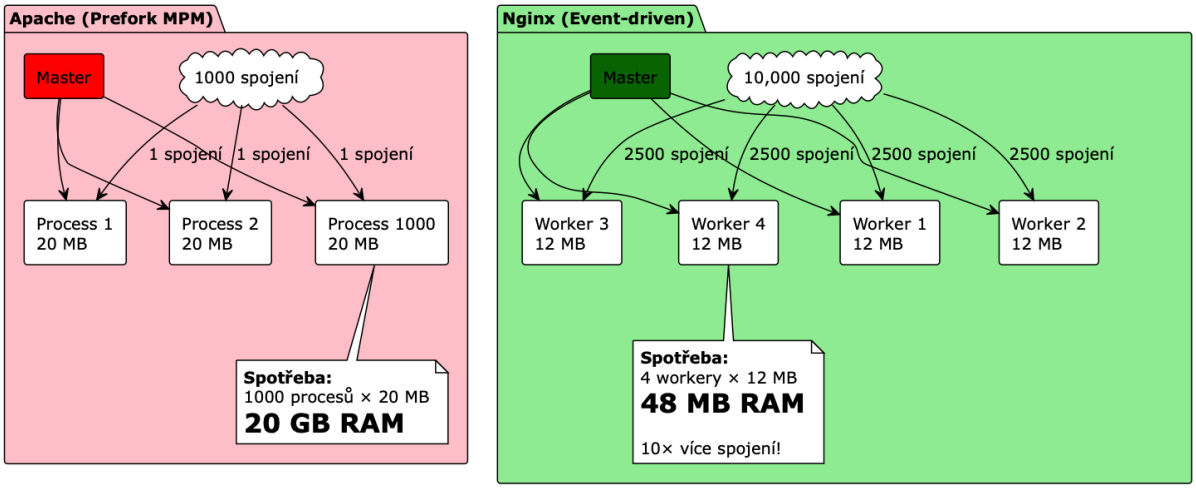
```
# Přesměrování HTTP -> HTTPS
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

# Odstranění .php z URL
RewriteCond %{REQUEST_FILENAME}.php -f
RewriteRule ^(.*)$ $1.php [L]

# Pretty URLs pro aplikaci
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?url=$1 [L,QSA]
```

- **moduly:**
  - Modulární architektura - funkce přidávány moduly
  - **Základní moduly:**
    - **mod_rewrite** - přepisování URL
    - **mod_ssl** - HTTPS podpora
    - **mod_headers** - manipulace s hlavičkami
    - **mod_proxy** - reverse proxy funkce
  - **PHP integrace:**
    - **mod_php:**
      - PHP interpreter v Apache procesu
      - Každý Apache proces má PHP připravené
      - Nízká latence - žádná komunikace přes síť/socket
    - **mod_fcgid / mod_proxy_fcgi** - FastCGI pro PHP-FPM
- **.htaccess soubory:**
  - Konfigurace per adresář
  - Umožňuje uživatelům měnit nastavení bez přístupu k hlavní konfiguraci
  - **Výhody:**
    - Flexibilita pro uživatele
    - Změny bez restartu serveru
    - Dobré pro sdílený hosting
  - **Nevýhody:**
    - Výkonnostní overhead (každý request = hledání **.htaccess** v celé cestě)
    - Bezpečnostní riziko (uživatel může přepsat nastavení)
    - Horší přehlednost konfigurace

Příklad .htaccess souboru

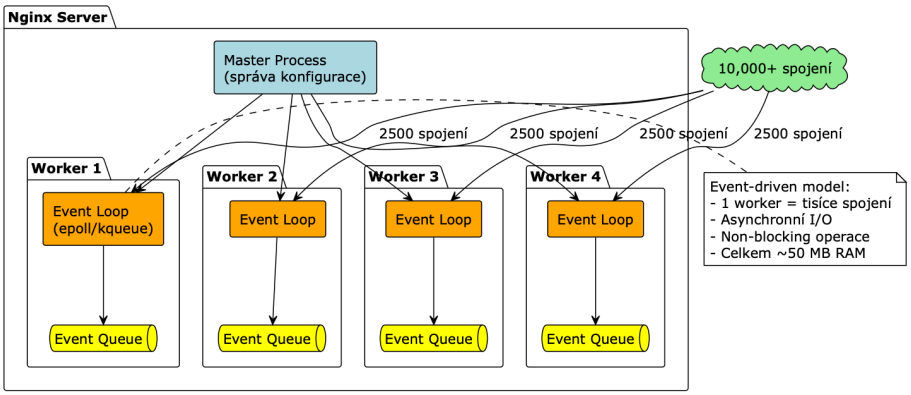
```
# Přesměrování na HTTPS
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}/$1 [R=301,L]

# Nastavení cache hlaviček
<FilesMatch "\.(jpg|jpeg|png|gif|css|js)$">
    Header set Cache-Control "max-age=604800, public"
</FilesMatch>

# Ochrana proti hotlinkingu
RewriteCond %{HTTP_REFERER} !^$
RewriteCond %{HTTP_REFERER} !^https?://(www\.)?example\.com [NC]
RewriteRule \.(jpg|jpeg|png|gif)$ - [F]
```

- **Nginx:**
  - Je řešením na **C10K** (concurrency 10000 connections) problem
  - Event-driven, asynchrony architektura
  - Jeden worker process = tisíce spojení
  - Nízká spotřeba paměti, vysoký výkon



Vizualizace C10K problem

- **Architektura:**
  - Event-driven, asynchronní, non-blocking
  - **<u>Master process:</u>**
    - Správa konfigurace
    - Spawn worker procesů
    - Reload bez downtime
  - **<u>Worker processes:</u>**
    - Zpracování požadavků
    - Event loop (epoll/kqueue)
    - jeden worker = tisíce spojení



- **Jak funguje Event loop:**
  1. Worker čeká na událost
  2. Když událost přijde, worker ji rychle zpracuje
  3. Pokud operace blokuje, worker pokračuje v jiné práci
  4. Až je operace hotová, worker se k ní vrátí
  - **Výhody:**
    - Žádné čekání na I/O - asynchronní operace
    - Minimální context switching
    - Nízká spotřeba paměti (žádné procesy per spojení)
    - Skvělý výkon při tisících souběžných spojení
- **Konfigurace:**
  - **hierarchická struktura nginx.conf:**

Main a Events context

```
# Main context
user www-data;
worker_processes auto;
pid /run/nginx.pid;

events {
    # Events context
    worker_connections 1024;
}
```

HTTP, Server a Location

```
http {
  include /etc/nginx/mime.types;
  default_type application/octet-stream;

  server {
    listen 80;
    server_name example.com;

    location / {
      root /var/www/html;
    }
  }
}
```

  - **Server block (ekvivalent Apache VirtualHost):**

Základní konfigurace

```
server {
listen 80;
listen [::]:80;  # IPv6
server_name www.example.com example.com;

root /var/www/example;
index index.html index.php;

# Logování
access_log /var/log/nginx/example-access.log;
error_log /var/log/nginx/example-error.log;
```

Location bloky

```
# Hlavní location
location / {
try_files $uri $uri/ /index.php?$query_string;
}

# PHP soubory
location ~ \.php$ {
fastcgi_pass unix:/var/run/php/php8.1-fpm.sock
fastcgi_index index.php;
include fastcgi_params;
}
}
```

- **Location (URL matching patterns):**

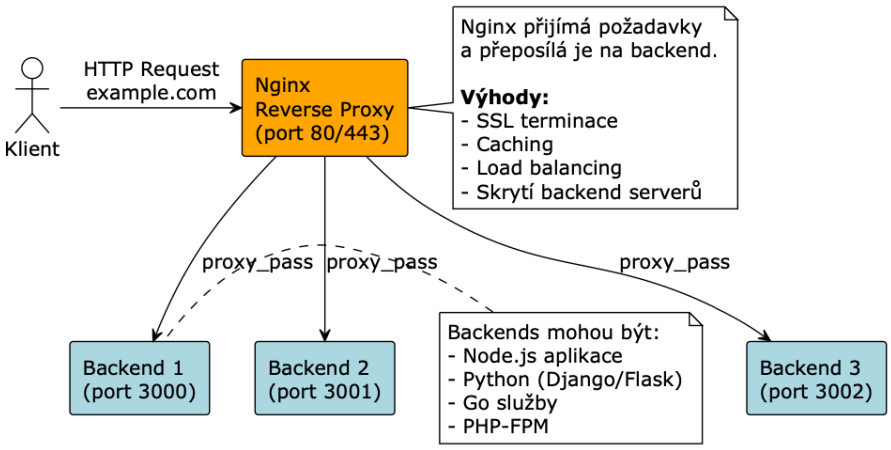
Exact, Prefix, Regex

```
# 1. Exact match (nejvyšší priorita)
location = / {
return 200 "Homepage";
}

# 2. Prefix match
location /images/ {
root /var/www/static;
}

# 3. Regex (case-sensitive)
location ~ \.php$ {
fastcgi_pass php-fpm;
}
```

Case-insensitive, Priority

```
# 4. Regex (case-insensitive)
location ~* \.(jpg|jpeg|png|gif)$ {
expires 30d;
}

# 5. Prefix s vyšší prioritou
location ^~ /api/ {
proxy_pass http://backend;
}
```

- **Reverse proxy (přeposílání požadavků na backend):**
  - Klient komunikuje jen s Nginx, backend je skrytý



Server a proxy_pass

```
server {
listen 80;
server_name api.example.com;

location / {
  # Přeposlání na backend
  proxy_pass http://localhost:3000;

  # Forwarding hlaviček
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
```

Hlavičky a timeouty

```
  proxy_set_header X-Forwarded-For $proxy_add_
  proxy_set_header X-Forwarded-Proto $scheme;

  # Timeouty
  proxy_connect_timeout 60s;
  proxy_send_timeout 60s;
  proxy_read_timeout 60s;
}
}
```

- **Load balancing:**
  - Rozdělení zátěže mezi více serverů

Upstream block s load balancingem

```
upstream backend {
  # Round-robin (default)
  server backend1.example.com;
  server backend2.example.com;
  server backend3.example.com;
}
server {
  listen 80;
  server_name example.com;
  location / {
      proxy_pass http://backend;
      proxy_set_header Host $host;
  }
}
```

- **Algoritmy:**
  - **Round robin:**
    - Výchozí, střídá servery rovnoměrně
    - pro stejně výkonné servery
  - **Least connections:**
    - Posílá na server s nemenším počtem aktivních spojení
    - pro různě dlouhé požadavky
  - **IP hash:**
    - Stejný klient vždy na stejný server
    - pro session persistence
  - **Weight:**
    - Servery s vyšší váhou dostávají více požadavků
    - pro různě výkonné servery

Round-robin a Least conn

```
# 1. Round-robin (default)
upstream backend {
    server backend1.example.com;
    server backend2.example.com;
}

# 2. Least connections
upstream backend {
    least_conn;
    server backend1.example.com;
    server backend2.example.com;
}
```

IP hash a Weight

```
# 3. IP hash (session persistence)
upstream backend {
    ip_hash;
    server backend1.example.com
    server backend2.example.com;
}

# 4. Weight (priorita)
upstream backend {
    server backend1.example.com weight=3;
    server backend2.example.com weight=1;
}
```

- **Nginx a statické soubory:**
  - Optimalizace servírování statických souborů:

Sendfile a TCP optimalizace

```
server {
  listen 80;
  server_name static.example.com;
  root /var/www/static;

  # sendfile (kernel space)
  sendfile on;
  tcp_nopush on;
  tcp_nodelay on;

  # Gzip komprese
  gzip on;
  gzip_types text/css application/javascript;
  gzip_min_length 1000;
```

Cache hlavičky

```
  # Cache pro statické soubory
  location ~* \.(jpg|jpeg|png|gif|ico|css|js)
      expires 1y;
      add_header Cache-Control "public, immuta
  }
}
```

Cachování

- **Cache** (vyrovnávací paměť) ukládá dříve získané odpovědi pro opakované použití. Tím snižuje zátěž serverů a zrychluje načítání stránek pro uživatele
- Čerstvost uložené odpovědi určují hlavičky jako **Cache Control** (např. s direktivou **max-age**) nebo dříve používaná hlavička **Expires**
- I prošlou (stale) odpověď v cache lze často obnovit (revidovat) pomocí podmíněného požadavku namísto opětovného stažení
- **Podmíněné požadavky:**
  - Obsahuje validační hlavičku (např. **If-Modified-Since** nebo **If-None-Match**), podle které server ověří, zda se daný zdroj od posledního poskytnutí změnil
  - Pokud se obsah nezměnil, server místo plné odpovědi pošle jen stavový kód **304 Not Modified** a žádné tělo - odpověď je tak velmi malá
  - HTTP/1.1 zavedlo entitní tag **ETag** jako alternativní silný validátor. Server posílá **ETag** v odpovědi a klient ho pak odešle zpět v hlavičce **If-None-Match**. Pokud se **ETag** shoduje, server opět vrátí jen **304 Not Modified**

```
location ~* \.(jpg|jpeg|png|gif|css|js)$ {
    expires 1y;
    add_header Cache-Control "public, max-age=31536000";
}
```

Cache pro statické soubory. Na rok se uloží a prohlížeč se vůbec neptá serveru a použije soubory z lokální cache.

- **Proxy cache:**
  - Cachování odpovědí z backendu

Nginx proxy cache konfigurace

```
# Definice cache (v http kontextu)
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api_cache:10m max_size=1g inactive
server {
    location /api/data/ {
        proxy_pass http://backend;
        # Použití cache
        proxy_cache api_cache;
        proxy_cache_valid 200 10m;  # Cache 200 odpovědí na 10 minut
        proxy_cache_valid 404 1m;   # Cache 404 na 1 minutu
        proxy_cache_key "$scheme$request_method$host$request_uri";
        # Hlavičky pro debugging
        add_header X-Cache-Status $upstream_cache_status;
    }
}
```